

PART OF A SERIES OF TUTORIALS ON QUANTUM ERROR CORRECTION

Quantum Error Correction

From parity checks to a quantum memory experiment

Revised 21 September 2026

In the previous tutorial, we introduced quantum computing as a way of processing information using the laws of quantum mechanics. We also discussed four central features of quantum information: superposition, interference, entanglement, and measurement. These ideas provide the foundation for understanding quantum error correction, which is essential for building large-scale, fault-tolerant quantum computers.

The primary goal of this tutorial is to introduce stabilizers, error syndromes, and decoding, using the three-qubit repetition code as a running example. We begin with ideal encoding and stabilizer measurements, and then add the complications that arise in practice: errors in the data, noisy measurement outcomes, and faults in the circuits used to extract the syndrome.

1 Error correction

When we store or transmit information, errors can happen. Physical noise can change the value read from a bit: a 0 becomes a 1, or a 1 becomes a 0. Our goal is to recover the original message despite these changes.

The basic strategy is *redundancy*: extra bits that do not carry new independent message content but give us a way to recognize and recover from some changes in the original message. Those extra bits might be copies, or they might record relationships among message bits.

The simplest example is the three-bit repetition code:

$$0 \mapsto 000, \quad 1 \mapsto 111. \tag{1}$$

The bit strings 000 and 111 are the valid *codewords*; each represents one legitimate message, 0 or 1. If we receive a string such as 010, which violates the code's pattern, this may indicate that an error has occurred. The map from the original bit to its codeword is called *encoding*. After transmission, we apply a *decoding* rule to the received word to infer the original bit. For this code, the decoding rule is majority voting. For example, for the received word 010, the majority value is 0, so we decode it as 0. Assuming that at most one bit has flipped, we infer that the middle bit was faulty. We can then correct the received word by flipping that bit, recovering the codeword 000, which represents the original message 0. None of the three bits is a guaranteed reference; we trust the majority because of our assumption about the error process.

If each bit flips independently with probability p , and all other operations are ideal, majority voting fails whenever at least two of the three bits are flipped:

$$P_{\text{fail}} = \binom{3}{2} p^2 (1 - p) + p^3 = 3p^2 - 2p^3. \tag{2}$$

For $0 < p < \frac{1}{2}$, we have $P_{\text{fail}} < p$. Thus, under this independent-error model, the three-bit repetition code reduces the probability of an incorrect decoded bit. At $p = \frac{1}{2}$, encoding provides no improvement; for $\frac{1}{2} < p < 1$, the decoded bit is more likely to be wrong than the original unencoded bit. The assumptions matter: for example, a disturbance that flips all three bits defeats majority voting. We will later consider errors in the checking and decoding processes themselves.

1.1 Parity: checking a relationship

How do we compare the copies? XOR, written $\oplus$, returns 0 when two bits agree and 1 when they disagree:

$$0 \oplus 0 = 1 \oplus 1 = 0, \quad 0 \oplus 1 = 1 \oplus 0 = 1.$$

For three bits $b_1 b_2 b_3$, define two parity checks:

$$s_1 = b_1 \oplus b_2, \quad s_2 = b_2 \oplus b_3. \quad (3)$$

The pair of check outcomes $s_1 s_2$ is called the *syndrome*. Both valid codewords, 000 and 111, give $s_1 s_2 = 00$. For a single-bit error, the syndrome identifies which check was violated. If bit 1 flips, only the first check changes, giving $s_1 s_2 = 10$. If bit 2 flips, both checks change, giving $s_1 s_2 = 11$. If bit 3 flips, only the second check changes, giving $s_1 s_2 = 01$, regardless of the original codeword being 000 or 111. Thus, the checks reveal where the bits disagree, but not whether the original message was 0 or 1.

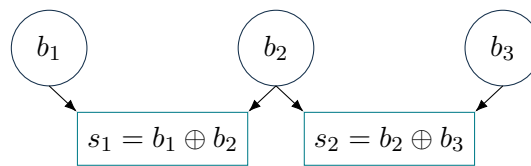


Figure 1: The middle bit participates in both checks. A single flip there changes both check outcomes. A boundary bit participates in only one check.

Replication and parity checks are not competing ideas: replication supplies redundancy, and parity checks test its structure. Other codes use parity bits instead of literal copies. For example, append $p = b_1 \oplus b_2 \oplus b_3$, so every valid encoded string satisfies $b_1 \oplus b_2 \oplus b_3 \oplus p = 0$. An odd total parity detects an error, but one check cannot locate it, and an even number of flips can go unnoticed. Carefully chosen overlapping checks can identify errors with less redundancy than repetition.

A code can be described by its valid messages, or by the constraints those messages must satisfy. This second viewpoint is the bridge to quantum error correction.

2 Quantum errors: bits, phases, and small rotations

For a qubit, preserving the values 0 and 1 is not enough. We must also preserve the amplitudes and their relative phase. The single-qubit Pauli operators provide a useful language for describing the most important errors:

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (4)$$

Their algebra includes

$$X^2 = Z^2 = I, \quad XZ = -ZX, \quad Y = iXZ. \quad (5)$$

The square of each Pauli operator becomes the identity; distinct nonidentity single-qubit Paulis anticommute; the product of two distinct nonidentity Pauli operators is proportional to the remaining one. Since Y is, up to a phase, a product of X and Z , we can use X and Z as the two basic error types. For $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$,

$$X|\psi\rangle = \alpha|1\rangle + \beta|0\rangle, \quad (6)$$

$$Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle. \quad (7)$$

We call X a bit flip and Z a phase flip, relative to the computational basis.

In the Bloch-sphere representation, each of X , Y , and Z acts as a 180° rotation—a half-turn—about its corresponding axis. In particular, X exchanges $|0\rangle$ and $|1\rangle$. The operator Y is, up to an overall phase, equivalent to applying Z and X in succession.

2.1 What if the error is only a small rotation?

Physical errors need not be exact half-turns. Since $X^2 = I$, expanding the exponential separates its even and odd powers:

$$R_X(\epsilon) = e^{-i\epsilon X/2} = \cos \frac{\epsilon}{2} I - i \sin \frac{\epsilon}{2} X. \quad (8)$$

A small rotation is a coherent linear combination of I and X , not necessarily a random choice between them. More generally, every single-qubit operator expands in I, X, Y, Z . This is why Pauli errors are useful building blocks even for continuous noise. Once we have a code and a syndrome measurement, we will show exactly how Eq. (8) can be corrected.

2.2 What about entangled, multi-qubit states?

An entangled state cannot be represented by separate pure-state Bloch spheres, but Pauli operators still work. On n qubits we use *Pauli strings*, such as $X \otimes I \otimes Z$, and every operator has an expansion

$$E = \sum_{P \in \{I, X, Y, Z\}^{\otimes n}} c_P P. \quad (9)$$

There are 4^n such basis operators. They act on entangled states just as on any other state: for example, $(X \otimes I)(|00\rangle + |11\rangle)/\sqrt{2} = (|10\rangle + |01\rangle)/\sqrt{2}$. For general open-system noise, each Kraus operator has a Pauli expansion; a general channel need not be a classical mixture of Pauli errors.

3 Defining a quantum code through stabilizers

A code can be described by its valid codewords or by the constraints they must satisfy. For a quantum code, superpositions of its basis codewords must also be valid. Error correction must preserve these superpositions: a measurement that identifies the logical codeword would collapse the encoded state and destroy the coherence we want to protect. We therefore check constraints shared by all valid code states. *Stabilizer codes* provide such checks, extending classical parity checks to quantum information and giving us the language used later for the surface code.

We choose mutually commuting Hermitian Pauli operators $S_1, \dots, S_r$, called *stabilizer generators*.¹ Their products, including the identity, form the *stabilizer group* $\mathcal{S} = \langle S_1, \dots, S_r \rangle$. The *codespace* consists of the states that every element of this group leaves unchanged:

$$S|\psi_L\rangle = |\psi_L\rangle \quad \text{for every } S \in \mathcal{S}. \quad (10)$$

Equivalently, it is their common $+1$ eigenspace. Measuring a generator returns $+1$ for a satisfied check and -1 for a violated check. Commutation makes the checks compatible. We also require $-I \notin \mathcal{S}$: otherwise the constraints would demand $-|\psi_L\rangle = |\psi_L\rangle$, which no nonzero state can satisfy. The identity I , in contrast, always belongs to the group. Stabilizer codes are one class of quantum codes, not the only possibility [1].

3.1 The repetition code as quantum parity checks

For a computational-basis bit b , $Z|b\rangle = (-1)^b|b\rangle$. Consequently,

$$(Z \otimes Z)|b_1b_2\rangle = (-1)^{b_1+b_2}|b_1b_2\rangle. \quad (11)$$

Thus ZZ has eigenvalue $+1$ when the two bits agree and -1 when they disagree. On three qubits, choose

$$S_1 = Z_1Z_2 = ZZI, \quad S_2 = Z_2Z_3 = IZZ, \quad \mathcal{S} = \{III, ZZI, IZZ, ZIZ\}. \quad (12)$$

Here Z_j means Z on qubit j and identity elsewhere; adjacent letters denote tensor products. There are two independent generators: the third nonidentity stabilizer, $ZIZ = S_1S_2$, adds no new constraint.

Both checks require adjacent bits to agree, so the only allowed computational-basis strings are 000 and 111 . We identify $|0_L\rangle = |000\rangle$ and $|1_L\rangle = |111\rangle$, where L denotes a *logical* qubit encoded across the three physical qubits. By linearity, every normalized superposition

$$|\psi_L\rangle = \alpha|000\rangle + \beta|111\rangle \quad (13)$$

passes both checks, whatever the amplitudes α and β .

A single X error breaks one or both of these parity constraints. A Z error commutes with both checks and can change the logical relative phase undetected. To detect phase flips instead, use the same construction in the X basis: the checks X_1X_2 and X_2X_3 stabilize all superpositions of $|+++\rangle$ and $---\rangle$. Since $Z|+\rangle = |-\rangle$ and $Z|-\rangle = |+\rangle$, a Z error now creates a disagreement. This phase-flip code has the complementary vulnerability to X errors. Protecting against both requires a different code with suitable compatible checks; we cannot simply combine these two sets, since some of their checks anticommute.

¹Hermitian means $A = A^\dagger$, where $\dagger$ is the conjugate transpose. Such operators represent observables and have real eigenvalues. Tensor products of I, X, Y, Z , with an overall sign ± 1 , are Hermitian; the full Pauli group also includes factors $\pm i$, which are not suitable as observables. A nonidentity Hermitian Pauli check has outcomes $+1$ and -1 .

4 Syndromes: reading the checks

The measured check outcomes form a *syndrome*, just as in the classical code. We record +1 as bit 0 and -1 as bit 1. A valid code state gives the trivial syndrome $00 \cdots 0$. To see how an error changes it, recall that two Pauli strings either commute or anticommute. For a Pauli error E ,

$$S_i E = (-1)^{s_i(E)} E S_i \implies S_i E |\psi_L\rangle = (-1)^{s_i(E)} E |\psi_L\rangle. \quad (14)$$

A commuting error leaves that check at +1; an anticommuting error changes it to -1. For example, X_2 anticommutes with both $Z_1 Z_2$ and $Z_2 Z_3$, giving syndrome 11. The complete single-flip pattern is compact:

$$I \longrightarrow 00, \quad X_1 \longrightarrow 10, \quad X_2 \longrightarrow 11, \quad X_3 \longrightarrow 01. \quad (15)$$

If at most one X error occurred before an ideal check round, the syndrome identifies it. Applying the same X_j again restores the encoded state because $X_j^2 = I$. The sign in Eq. (14) depends on the error and the check, not on α or β : we learn about the error without learning the message.

4.1 What the syndrome does not tell us

A *decoder* uses the syndrome and an error model to choose a recovery. The syndrome alone need not identify a unique error: X_1 and $X_2 X_3$ both give 10. If we assume a single flip and apply X_1 when the actual error was $X_2 X_3$, the remaining operation is $X_1 X_2 X_3$. It exchanges the two valid codewords, so the checks pass even though the logical information has changed.

Likewise, syndrome 00 means that no check was violated, not that no error occurred. Recovery succeeds when the combined error and correction leave *every* encoded state unchanged, up to a global phase. We do not need to identify the exact physical error if several possible errors have the same effect on the encoded information. To make this distinction precise, we need logical operators.

5 Logical operators: changing the encoded information

Stabilizers leave every code state unchanged. A *logical operator* keeps the state inside the codespace while acting on the encoded information. For a Pauli operator L that commutes with every check,

$$S_i L |\psi_L\rangle = L S_i |\psi_L\rangle = L |\psi_L\rangle. \quad (16)$$

Thus the state still passes every check. If L is a stabilizer, it does nothing to the encoded information. If it is not proportional to a stabilizer, it implements a nontrivial logical Pauli.

For the repetition code, we choose

$$\bar{X} = X_1 X_2 X_3, \quad \bar{Z} = Z_1. \quad (17)$$

The first exchanges $|0_L\rangle$ and $|1_L\rangle$; the second leaves $|0_L\rangle$ unchanged and multiplies $|1_L\rangle$ by -1 . They act just like X and Z on one qubit. Both commute with the stabilizers, but they anticommute with each other: $\bar{X}\bar{Z} = -\bar{Z}\bar{X}$.

Fixing one particular code state is not enough to make an operator a stabilizer. For example, Z_1 fixes $|0_L\rangle$, but changes $(|0_L\rangle + |1_L\rangle)/\sqrt{2}$. The distinction is its action on the *whole codespace*. Logical representatives are also not unique: $Z_2 = Z_1 S_1$, so Z_1 and Z_2 have the same logical action. More generally, multiplying by a stabilizer does not change that action.

Could we choose Z_1 as an additional stabilizer? Yes, but then only multiples of $|000\rangle$ pass all three checks. We have fixed the logical value and lost the freedom to encode an arbitrary qubit. This gives us a way to count how much information a code can hold.

5.1 Counting information and protection: $[[n, k, d]]$

An n -qubit register has 2^n independent basis states. Each independent, consistent Pauli check halves the dimension available to the code. With r independent generators, the remaining dimension is 2^{n-r} , enough to encode $k = n - r$ logical qubits:

$$\dim \mathcal{C} = 2^{n-r} = 2^k, \quad r = n - k. \quad (18)$$

Our two checks on three qubits therefore leave one logical qubit. In general, the stabilizer group has 2^{n-k} elements, while the logical Pauli operators are generated by k pairs $(\bar{X}_j, \bar{Z}_j)$. Each pair anticommutes; operators for different logical qubits commute. Ignoring global phases and identifying representatives that differ by a stabilizer gives 4^k logical Pauli classes.

To describe protection as well as capacity, we write $[[n, k, d]]$. Here n counts physical data qubits, k counts encoded logical qubits, and the *distance* d is the smallest number of qubits on which a Pauli error can act to make an undetected logical change:

$$d = \min_{L \text{ nontrivial logical Pauli}} \text{wt}(L). \quad (19)$$

The weight counts nonidentity factors: $\text{wt}(XII) = 1$ and $\text{wt}(XZX) = 3$. A low-weight stabilizer does not reduce the distance because it leaves the encoded information unchanged.

Two Pauli errors, each affecting at most t qubits, differ by an operator of weight at most $2t$. If $2t < d$, they cannot have both the same syndrome and different logical effects. This is why a distance- d code, with ideal recovery, corrects arbitrary errors on up to $t = \lfloor (d-1)/2 \rfloor$ qubits [2].

The repetition code is $[[3, 1, 1]]$: Z_1 is already a weight-one logical error. Its protection against one X error does not imply protection against every single-qubit error. The smallest purely X logical operator has weight three, explaining that restricted protection. Measurement ancillas are additional hardware and are not counted in n .

6 Returning to small rotations

We can now answer the question from Eq. (8). A small rotation on the first data qubit gives

$$R_{X_1}(\epsilon)|\psi_L\rangle = \cos \frac{\epsilon}{2} |\psi_L\rangle - i \sin \frac{\epsilon}{2} X_1 |\psi_L\rangle. \quad (20)$$

The two terms have different syndromes. Ideal check measurements return 00 with probability $\cos^2(\epsilon/2)$, leaving the original encoded state, or 10 with probability $\sin^2(\epsilon/2)$, leaving $X_1|\psi_L\rangle$ up to a global phase. In the latter case, applying X_1 restores the state. We do not need to estimate the rotation angle.

This works because both components are correctable. A rotation generated by $Z_1 = \bar{Z}$, however, changes the logical relative phase while leaving every check satisfied. More generally, a code that corrects a set of errors also corrects their linear combinations. Syndrome measurement separates components with different syndromes; it does not imply that the original noise was a random choice of Pauli errors [3].

We have described what the checks reveal. Next we need circuits that obtain that information while preserving the encoded superposition.

7 Implementation: a three-qubit quantum memory

We have described how the repetition code detects and corrects a single bit flip. Now we want to implement this protection: prepare one logical qubit, store it while repeatedly checking for errors, and finally read out its logical value. Our three data qubits are checked by $S_1 = Z_1 Z_2$ and $S_2 = Z_2 Z_3$. Additional qubits, called *ancillas*, let us read these checks without directly measuring the encoded information.

7.1 From CNOT gates to a parity check

The controlled-NOT gate, or CNOT, has a control qubit c and a target qubit t . It flips the target when the control is $|1\rangle$:

$$\text{CNOT}_{c \rightarrow t} |x\rangle_c |y\rangle_t = |x\rangle_c |y \oplus x\rangle_t. \quad (21)$$

Its action on superpositions follows by linearity. In circuit diagrams, the filled dot marks the control and the circled plus marks the target; time runs from left to right.

To measure $Z_1 Z_2$, prepare an ancilla in $|0\rangle$, apply a CNOT from each data qubit to the ancilla, and then measure the ancilla in the computational basis:

$$|b_1 b_2\rangle |0\rangle_a \xrightarrow{\text{CNOT}_{1 \rightarrow a}} |b_1 b_2\rangle |b_1\rangle_a \xrightarrow{\text{CNOT}_{2 \rightarrow a}} |b_1 b_2\rangle |b_1 \oplus b_2\rangle_a. \quad (22)$$

Measuring the ancilla in the computational basis then gives $m = b_1 \oplus b_2$, corresponding to the check eigenvalue $\lambda = (-1)^m$.

The shared ancilla implements the selective readout we need: 00 and 11 leave it in the same state, as do 01 and 10. Measuring it distinguishes the two parities while preserving superpositions within each parity sector. Both CNOTs must precede the measurement; reading the ancilla after only the first would reveal the individual value b_1 .

The $Z_2 Z_3$ check uses the same circuit with data qubits 2 and 3. Together, the two check results form one syndrome round; each ancilla must be prepared in $|0\rangle$ before its next use.

We can also track the measured observable. With $U = \text{CNOT}_{2 \rightarrow a} \text{CNOT}_{1 \rightarrow a}$,

$$U^\dagger Z_a U = Z_1 Z_2 Z_a. \quad (23)$$

Since the input ancilla has $Z_a = +1$, measuring its output reads the data observable $Z_1 Z_2$. This is the Heisenberg viewpoint: follow the observable through the circuit rather than the state.

7.2 Detecting phase errors with an X check

The same idea detects phase errors, but the check basis is reversed. A Z error changes a relative phase and can commute with the Z checks of the bit-flip repetition code. To detect it, use the phase-flip code with checks $X_1 X_2$ and $X_2 X_3$. A Z error anticommutes with every X check that contains its qubit, so it changes the corresponding syndrome signs.

The encoded state here is $\alpha|+++\rangle + \beta|---\rangle$. This is a different encoding from $\alpha|000\rangle + \beta|111\rangle$: we have changed the basis of the entire code, not added an incompatible check to the old code.

The two circuits make the basis reversal visible. A Z check detects bit flips: the data qubits control CNOTs into a $|0\rangle$ ancilla, which is then read in Z . An X check detects phase flips: a $|+\rangle$ ancilla controls the CNOTs into the data, and is read in X . In both cases the ancilla reveals only the two-qubit parity.

To measure $X_1 X_2$, prepare an ancilla in $|+\rangle$, use it as the control of CNOTs targeting data qubits 1 and 2, and measure the ancilla in the X basis. If U is this two-CNOT circuit, then

$$U^\dagger X_a U = X_a X_1 X_2.$$

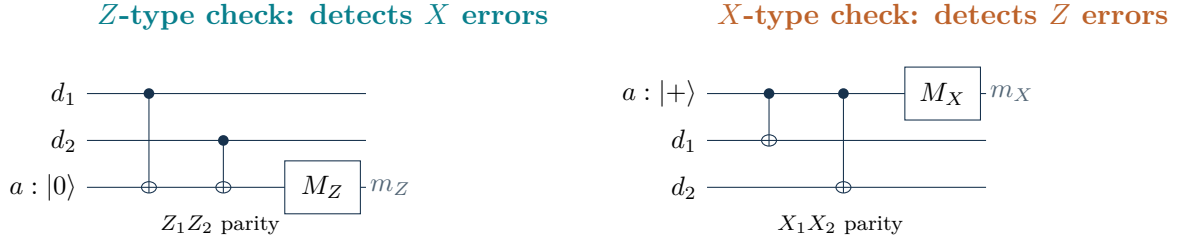


Figure 2: The same parity-measurement pattern in two complementary bases. Filled dots are CNOT controls and circled pluses are targets. The direction of each CNOT, the prepared ancilla state, and the readout basis all reverse together.

Because the input ancilla has $X_a = +1$, its final X readout measures $X_1 X_2$. Equivalently, apply H to the ancilla and measure it in the computational basis. The syndrome table is the phase analogue of the one above:

$$I \longrightarrow 00, \quad Z_1 \longrightarrow 10, \quad Z_2 \longrightarrow 11, \quad Z_3 \longrightarrow 01.$$

For example, Z_2 anticommutes with both $X_1 X_2$ and $X_2 X_3$, so both check outcomes flip. Applying the inferred Z_2 restores the encoded state. The bit-flip and phase-flip repetition codes are complementary; their checks cannot simply be combined because some X -type and Z -type checks would anticommute. A full quantum code must choose a larger compatible set of checks, as the surface code does.

7.3 Preparing the logical state

To encode $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, initialize the other two data qubits in $|0\rangle$ and apply CNOTs from qubit 1 to qubits 2 and 3:

$$\begin{aligned} (\alpha|0\rangle + \beta|1\rangle)|00\rangle &\xrightarrow{\text{CNOT}_{1\rightarrow 2}} \alpha|000\rangle + \beta|110\rangle \\ &\xrightarrow{\text{CNOT}_{1\rightarrow 3}} \alpha|000\rangle + \beta|111\rangle = |\psi_L\rangle. \end{aligned} \tag{24}$$

All three output qubits now store the logical information. This does not create three independent copies of $|\psi\rangle$; it encodes the state in their joint state.

For a first memory experiment, we can prepare $|0_L\rangle$ or $|1_L\rangle$ simply by preparing $|000\rangle$ or $|111\rangle$. To prepare $|+_L\rangle$, use $|+\rangle$ as the input to the encoding circuit. We initially assume ideal preparation; the encoding circuit alone does not guarantee protection against faults during preparation.

7.4 Noisy checks and a memory experiment

In practice, the checking circuit can also fail. An ancilla readout may report the wrong bit, and a faulty gate may affect the data, the ancilla, or both. One unexpected syndrome therefore need not mean that a data qubit flipped. We repeat the checks and use their history to distinguish likely explanations.

First consider a simple model: data X errors occur between ideal rounds, and reported syndrome bits can be wrong. If no corrections are applied between rounds, a persistent parity change might produce 0, 0, 1, 1, 1, whereas one bad report might produce 0, 0, 1, 0, 0. The difference is visible in the changes between consecutive results:

$$\delta_i(t) = \tilde{s}_i(t) \oplus \tilde{s}_i(t-1), \tag{25}$$

where $\tilde{s}_i(t)$ is the recorded result of check i in round t . This parity is a detector outcome; a nonzero outcome is a *detection event*. A decoder uses their pattern across checks and time, together with an error model, to infer a recovery. A detailed circuit-noise model must also account for faults during the CNOTs.

A memory experiment follows this sequence: prepare a logical state, store it through T rounds of checks, then measure the data and decode the complete record. We need not physically correct the data after every round; inferred Pauli corrections can be tracked classically and included when interpreting the final readout.

7.5 Reading the logical information

At the end of storage or computation, we deliberately read the encoded information. The measurement basis selects a logical observable; this final readout need not preserve the encoded state.

For logical Z , measure all three data qubits in Z . With ideal checks and at most one flip in the final three-bit record, majority voting gives the logical bit. With noisy repeated checks, the decoder uses the syndrome history together with these final outcomes. For an undamaged state $\alpha|0_L\rangle + \beta|1_L\rangle$, the logical outcomes 0 and 1 have probabilities $|\alpha|^2$ and $|\beta|^2$.

For logical X , use $\bar{X} = X_1X_2X_3$: measure each data qubit in X and multiply the three eigenvalue outcomes. Equivalently, apply H to each qubit before a Z measurement. If the resulting bits are b_1, b_2, b_3 , the logical eigenvalue is

$$\bar{x} = (-1)^{b_1+b_2+b_3}. \quad (26)$$

Unlike the majority-vote readout, one incorrect individual outcome flips this product. The repetition code does not protect both logical observables equally.

To assess the memory, repeat the experiment and compare the decoded logical result with the prepared value. Preparing and reading only $|0_L\rangle$ and $|1_L\rangle$ tests preservation of a logical bit; it does not establish preservation of an arbitrary quantum state. The undetected phase errors of this code remain the reason we need codes that protect against both bit and phase errors.

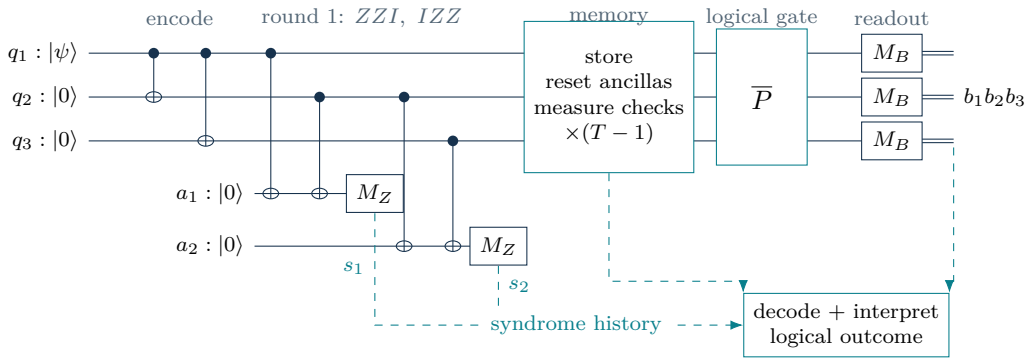


Figure 3: From encoding to logical readout. One explicit check round and $T - 1$ boxed rounds give T rounds in total. Choose $\bar{P} = I$ for a memory experiment, or apply a known logical $\bar{X}$ or $\bar{Z}$. M_B measures in $B = Z$ or X ; decoding uses the full record and accounts for $\bar{P}$. Double wires and dashed arrows carry classical results. Crossings without dots are not gates.

8 Where this leads: Shor's code and the surface code

Shor's nine-qubit code is one way to combine bit- and phase-error protection. It corrects any single-qubit error under ideal recovery and provides an optional next example. The next tutorial in this series instead develops the surface code directly.

We then turn to the surface code, where geometry becomes central. Local X -type and Z -type checks are arranged across a two-dimensional layout. That arrangement determines which errors affect which checks, how logical operators cross the code, and how distance grows with its size. The same stabilizer and syndrome language will let us understand this geometric construction.

Questions to check your understanding

1. Why do parity checks of a repetition code reveal disagreement without revealing the original bit?
2. Why do commuting check operators need the additional condition $-I \notin \mathcal{S}$?
3. Why is Z_1 logical rather than a stabilizer, even though it fixes $|0_L\rangle$?
4. Why does a small X_1 rotation have correctable syndrome components, while a small Z_1 rotation does not in this code?
5. How does measuring an ancilla implement a ZZ measurement? Which observable do you obtain by tracking its readout backward?
6. What distinguishes a persistent data fault from one bad syndrome report in an idealized time history?
7. What can a logical- Z repetition-code memory experiment establish, and what does it leave untested?

References

- [1] IBM Quantum Learning. [Stabilizer codes](#). Definition, consistency conditions, and repetition-code examples.
- [2] D. Gottesman (1997). [Stabilizer Codes and Quantum Error Correction](#). PhD thesis, arXiv:quant-ph/9705052. A general reference for stabilizer groups, logical operators, and distance.
- [3] IBM Quantum Learning. [Correcting quantum errors: introduction](#). Error discretization and the transition to full quantum protection.
- [4] IBM Quantum Documentation. [Repetition codes](#). Ancilla-based syndrome extraction and a memory experiment; note the limits of computational-basis benchmarking.